



Lab4 实验报告：RISC-V 目标代码生成

1. 实现思路

本实验基于 Lab 3 生成的 Zero IR，将中间代码翻译为 RISC-V 32 汇编。整体分三个阶段：指令选择、寄存器分配、汇编输出。

指令选择 采用一对一映射模式。InstSelector 遍历每个基本块的 IR 指令，将其转换为对应的 ASM 指令。LoadImm → li，Assign → mv，Binary → 依 op 生成

add/sub/mul/div/rem/slt，Unary 的 Neg 用 sub rd, zero, rs 取负，Lnot 用 sltiu 即 seqz，If → 依条件选 beq/bne/blt/bge，Call/Arg/Param 分别处理寄存器传参和栈传参，Load/Store 处理内存读写，Dec 调用 alloc_temp 在栈上分配空间并返回 sp+offset 地址，全局变量收集后在 emitter 输出 .data 段。

由于 IR 操作数既有变量名也有 #k 格式的立即数，处理二元运算时先检测 y[0]=='#' 和 z[0]=='#'，对立即数操作数用 new_temp() 生成临时虚拟寄存器并 li 加载，再按寄存器操作数处理。

寄存器分配 采用朴素 spill-all-to-stack 策略。遍历函数所有指令收集虚拟寄存器名，每个分配 4 字节栈槽。然后对每条指令：虚拟 use 前插入 lw tN, offset(sp) 从栈加载，替换指令内虚拟寄存器为 tN；虚拟 def 后插入 sw tN, offset(sp) 存回栈。t0/t1/t2 三个临时物理寄存器轮转使用。

汇编输出 在 ASMEmitter 中完成。每个函数输出 .globl 声明，prologue 调整 sp，保存 ra/fp，设置 fp，遍历基本块，epilogue 恢复 ra/fp + 恢复 sp + ret。全局变量输出 .data 段，use_venus 模式嵌入 read/write 系统调用。大帧超过立即数 12 位范围时使用 lui + add + lw/sw 分解。

2. 难点与亮点

2.1 难点

BinaryOp 与 Arith::Op 的关系运算转换。实验初期尝试用 static_cast 直接映射，但两者值域不对齐：Gt 需要交换操作数，Le/Ge/Eq/Neq 各需两条 RISC-V 指令组合（先 slt/sub，再 sltiu/sltu 取反）才能产生 0/1 值。最终用 switch 对每种 BinaryOp 逐一分派。

IR 的 Arg 节点 k 值为 0 。 IR translator (Lab 3 遗留代码) 创建 Arg 时未传 k ，所有参数的 node->k 默认为 0 。直接使用会导致多参数函数调用时全部参数进入 a0 。解决方式：在 InstSelector 中增加 arg_index 计数器，selectCall 时重置，selectArg 使用计数器序号而非 node->k 。

DEC 分配的栈空间与 outgoing 参数区重叠 。两者都从 sp+0 开始分配，当函数有 >8 个参数时，outgoing arg 写入会覆盖局部变量地址指针 。解决：在进入函数指令选择前预先调用 alloc_temp(128) 预留 outgoing 区，使 DEC 分配从 sp+128 开始，避免冲突 。

函数标签重复定义 。selectFunction 和 emitter prologue 都输出了函数标签 。修正为 selectFunction 返回空代码，由 emitter 统一在 prologue 中输出标签 。

大栈帧立即数溢出与优化 。大数组导致帧超 4KB，addi、lw/sw 的 12 位立即数/偏移不够 。最初用 li t3, N; add t3, rs, t3; lw rd, 0(t3) 分解，但 li 是一条伪指令，展开为 lui + addi，实际每条分解产生 4 条真实指令 。优化为直接使用 lui t3, hi; add t3, rs, t3; lw rd, lo(t3)，将 4 指令降为 3 指令 。在此基础上进一步优化帧布局：将 DEC 大分配从 fp 向下放置 (高位)，虚拟寄存器溢出保持在 sp 向上 (低位) 。优化前，大数组将虚拟寄存器偏移推到 40000+；优化后虚拟寄存器全部在 sp+0~9300 范围，sp 正偏移访问全部无需分解，仅负偏移 (DEC 取地址) 需 li 处理 。以 far_label 测试为例，优化前执行 71253 步，帧布局优化 + lui 分解后降至 55711 步，减少约 22% 。

2.2 亮点

指令选择中对 IR 操作数的统一处理 (变量名与立即数的自动检测和标准化) 、selectArg / selectParam 分别处理 k<8 寄存器传参和 k≥8 栈传参、selectBinary 用 slt + sltiu 等组合实现全部 6 种关系运算产生 0/1 值、大立即数 lui 分解及帧布局分离优化 。

3. 心得体会

本次实验从 IR 到 RISC-V 汇编的转换让我直观理解了编译器后端的工作流程 。最深的体会是：汇编代码生成不是简单地把 IR "翻译"成另一种格式，而是需要解决三个环环相扣的问题——指令选择决定了"做什么"，寄存器分配决定了"数据放哪里"，栈帧管理决定了"函数调用怎么衔接" 。任何一个环节的细节错误 (如参数序号、偏移量符号、栈空间重叠) 都会导致整个程序崩溃或结果错误 。

调试过程中反复查看 Venus 生成的汇编代码和错误信息，让我对 RISC-V 的寄存器约定、栈帧

布局 and 调用规范有了更具体的认识。朴素 spill-all 策略虽然简单，但让我理解了为什么需要更高级的寄存器分配算法——大量不必要的 lw/sw 严重拖慢执行速度。

4. 附加内容

借鉴声明

参考了课程提供的 Lab 4 文档和 Zero IR 模板代码。未复制他人代码。

AI 工具使用记录

- Prisma Agent (Claude Code)：辅助代码审查，指出实现错误（如 selectBinary 的 static_cast 问题、selectArg 的 k=0 问题、DEC 与 outgoing 区重叠问题、函数标签重复问题、大立即数分解优化、帧布局优化），但不直接生成实现代码，由本人根据提示独立完成。
- 相关的 AI 对话记录和 update notes 保留于仓库 reports/appends/lab4/ 目录。

lab4



杨思行

06/09 14:57

现在cp-3230104703这个文件夹下的项目git上出现了一些问题

```
(base) 🐉 base  yangsmac@YangsdeMacBook-Air  ~/Desktop/compiler  cd cp-3230104703
```

```
(base) 🐉 base  yangsmac@YangsdeMacBook-Air  ~/Desktop/compiler/cp-3230104703  lab3  git merge upstream/template/lab4-zero
```

自动合并 src/main.cpp

展开全部



deepseek-v4-pro

06/09 15:05

执行过程: 21次工具调用, 19条消息 > 1 1 1 1 1 +1

前置工作已完成。总结一下做了什么:

问题原因: 你在 lab3 分支上执行了 `git merge upstream/template/lab4-zero`, 正确做法应该是在 lab3 基础上创建 lab4 分支, 再在 lab4 上合并。

已完成的修复:

分支	状态
lab4	已创建 (含 lab4-zero 模板代码 + 冲突已解决), 已推送到 origin/lab4
lab3	已回退到合并前 (a2b722e add: lab3 report), 保持干净

lab4 分支现在包含了所有 lab4 代码生成相关的文件 (src/codegen/ 下的 `asm.hpp`、`asm_emitter`、`inst_selector`、`reg_allocator`, 以及 `src/analysis/control_flow`), 可以开始实验了。

7m 17s 1 1 1 1 1



杨思行

06/09 15:05

切回 lab4 分支开始干活



deepseek-v4-pro

06/09 15:15

执行过程: 37次工具调用, 27条消息 > 1 1 1 1 1 +6

nifty-dancing-crab.md 1 asm.hpp



杨思行

06/09 15:16

lab4.md

你不要修改代码, 我的初衷是让你引导我完成这个实验, 而不是让你直接生成



deepseek-v4-pro

06/09 15:16





deepseek-v4-pro

06/09 15:16

执行过程: 1 条消息

明白了, 我接下来切换到引导模式, 不再直接修改代码。

先梳理一下当前的状态和接下来的工作方向。你已经有了 lab4-zero 模板代码, 现在需要完成以下模块:

当前待实现清单

模块	文件	要做的事
ASM 指令定义	asm.hpp	ArithImm、La、Branch 还是骨架: Arith::Op 已补全 Mul/Div/Rem/Slt/Sltu
栈帧分配	c: control_flow.cpp	alloc_temp() / alloc_reg()
指令选择	.hpp	9 个 select* 是 #warning 空壳, 还缺 if/Dec/Load/Store 等节点
寄存器分配	c: reg_allocator.cpp	空壳
汇编输出	c: asm_emitter.cpp	prologue/epilogue 缺失

在继续之前想确认一下: 你引用的是 Accipit IR 的映射表, 但你 merge 的是 template/lab4-zero (Zero IR 模板)。你是想继续用 Zero IR, 还是想切到 Accipit IR?

16.5s



杨思行

06/09 15:17

继续用 Zero IR



deepseek-v4-pro

06/09 15:17

执行过程: 2 次工具调用, 3 条消息

好的, 我切换到引导模式。先看一下 Zero IR 的完整映射表, 这是你实现指令选择时的直接参考。

Zero IR → RISC-V 指令映射

中间代码	目标代码	模板状态
a = b + c	add reg(a), reg(b), reg(c)	selectBinary 空壳
a = b - c	sub reg(a), reg(b), reg(c)	同上
a = b * c	mul reg(a), reg(b), reg(c)	需新增 Arith::Op::Mul
a = b / c	div reg(a), reg(b), reg(c)	需新增 Arith::Op::Div
a = b % c	rem reg(a), reg(b), reg(c)	需新增 Arith::Op::Rem
a = b + #t	addi reg(a), reg(b), t	需实现 ArithImm
a = b - #t	addi reg(a), reg(b), -t	同上

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

杨思行

继续用 Zero IR

<code>a = b - c</code>	<code>sub reg(a), reg(b), reg(c)</code>	同上
<code>a = b * c</code>	<code>mul reg(a), reg(b), reg(c)</code>	需新增 <code>Arith::Op::Mul</code>
<code>a = b / c</code>	<code>div reg(a), reg(b), reg(c)</code>	需新增 <code>Arith::Op::Div</code>
<code>a = b % c</code>	<code>rem reg(a), reg(b), reg(c)</code>	需新增 <code>Arith::Op::Rem</code>
<code>a = b + #t</code>	<code>addi reg(a), reg(b), t</code>	需实现 <code>ArithImm</code>
<code>a = b - #t</code>	<code>addi reg(a), reg(b), -t</code>	同上
<code>a = b</code>	<code>mv reg(a), reg(b)</code>	已完成
<code>a = #t</code>	<code>li reg(a), t</code>	<code>selectLoadImm</code> 空壳
<code>a = -b</code>	<code>sub reg(a), zero, reg(b)</code>	<code>selectUnary</code> 空壳
<code>LABEL l:</code>	<code>l:</code>	<code>selectLabel</code> 空壳
<code>GOTO l</code>	<code>j l</code>	<code>selectGoto</code> 空壳
<code>IF x op y GOTO l</code>	<code>b<op> reg(x), reg(y), l</code>	缺 <code>selectIf</code>
<code>FUNCTION f:</code>	<code>f:</code>	<code>selectFunction</code> 空壳
<code>x = CALL f</code>	<code>call f; mv reg(x), a0</code>	<code>selectCall</code> 空壳
<code>ARG x</code>	<code>mv a_k, reg(x)</code>	<code>selectArg</code> 空壳
<code>RETURN x</code>	由 emitter 处理	保持空
<code>x = *y</code>	<code>lw reg(x), 0(reg(y))</code>	缺 <code>selectLoad</code>
<code>*x = y</code>	<code>sw reg(y), 0(reg(x))</code>	缺 <code>selectStore</code>
<code>x = &y</code>	<code>la reg(x), y</code>	缺 <code>selectLoadAddr</code>
<code>DEC x #k</code>	栈分配 + <code>addi reg(x), sp, off</code>	缺 <code>selectDec</code>
<code>GLOBAL x: #s</code>	<code>.data</code> 段输出	缺 <code>selectGlobal</code>
<code>PARAM x</code>	从 <code>a(k)</code> 或栈加载	缺 <code>selectParam</code>

建议从最简单的开始，建立信心。第一个实现 `selectLoadImm`：

C++

复制

```
// a = #t → li reg(a), t
ASM::Code InstSelector::selectLoadImm(const IR::LoadImmPtr& node) {
    ASM::Code code;
    code.push_back(ASM::Li::create(ASM::Reg(node->x), node->k));
    return code;
}
```

这只需要一行 `ASM::Li::create`，`Li` 类已经在 `asm.hpp` 里实现了。你觉得可以从这里开始吗？还是想先按别的顺序来做？

19.0s





杨思行

06/09 17:03

我执行make && make clean之后有这样一个错误src/codegen/inst_selector.cpp:19:12: error: no member named 'asm_code' in 'BasicBlock'
19 | block->asm_code = select(block->ir_code);, 请你看一下这个问题, 检查一下上面两个函数的实现并告诉我后续的工作



deepseek-v4-pro

06/09 17:04

› 执行过程: 2 次工具调用, 4 条消息

之前指出的两个问题已经修好了, 没有其他问题。

当前唯一的编译错误就是 BasicBlock 缺 asm_code 字段。在 control_flow.hpp 第 18 行后加上:

C++

复制

```
ASM::Code asm_code;
```

加在 IR::Code ir_code; 下面即可。

修好之后, 下一波实现 selectCall 和 selectArg。回顾一下:

selectCall — IR Call 有 node->func (函数名) 和 node->x (返回值变量, 空串 = void) :

C++

复制

```
// 1. 先 emit call  
code.push_back(ASM::Call::create(node->func));  
// 2. 如果有返回值, emit mv reg(x), a0  
if (!node->x.empty()) {  
    code.push_back(ASM::Mv::create(ASM::Reg(node->x), ASM::Reg::a0));  
}
```

selectArg — IR Arg 有 node->x (变量名) 和 node->k (从 0 开始的参数序号) :

C++

复制

```
// ARG x → mv a(k), reg(x)  
code.push_back(ASM::Mv::create(  
    ASM::Reg("a" + std::to_string(node->k)),  
    ASM::Reg(node->x)));
```

Reg("a0") 到 Reg("a7") 会被构造函数自动识别为物理寄存器, 后续寄存器分配阶段不会对它们做 spill。修完 asm_code 后可以一并写好这两个继续推进。

21.3s



杨思行

06/09 17:06

我已经都实现了, 你检查一下对不对



deepseek-v4-pro

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)



block->asm_code = select(block->if_code);, 请你看一下这个问题。检查一下上面两个函数的实现并告诉我后续的工作

06/09 17:06

执行过程: 2 次工具调用, 2 条消息

三个都正确。

- asm_code 字段位置对 ✓
- selectCall: call 指令 + 有返回值时 mv a0 ✓
- selectArg: mv a(k), reg(x) ✓

继续推进。还剩 selectBinary 和 selectUnary 这两个空壳, 这是指令选择里最核心也最复杂的。

selectUnary 相对简单, 三种情况:

IR	RISC-V
x = -y (Neg)	sub reg(x), zero, reg(y)
x = +y (Pos)	mv reg(x), reg(y)
x = !y (Lnot)	sltiu reg(x), reg(y), 1 (即 seqz: y==0 时 x=1)

用 node->op 判断, 对应 UnaryOp::Neg / UnaryOp::Pos / UnaryOp::Lnot。

selectBinary 需要先处理一件事: IR 的 Binary 节点中, node->y 和 node->z 可能是变量名 (如 "T5"), 也可能是立即数 (如 "#3", 以 # 开头)。你需要一个辅助方法 new_temp() 生成临时虚拟寄存器名 (如 "@tmp0")。

核心思路: 对每个操作数, 如果是立即数 (y[0] == '#'), 先 emit 一条 li 把它加载到临时寄存器, 再当作寄存器操作数处理:

Code

复制

```
y_is_imm = (y[0] == '#')
z_is_imm = (z[0] == '#')

if y_is_imm: tmp = new_temp(); li tmp, stoi(y.substr(1)); y = tmp
if z_is_imm: tmp = new_temp(); li tmp, stoi(z.substr(1)); z = tmp
```

然后按 node->op 选择操作:

BinaryOp	RISC-V
Add	add
Sub	sub
Mul	mul
Div	div
Mod	rem
Lt	slt - x = (y < z)
Gt	slt reg(x), reg(z), reg(y) - 交换操作数
Le	slt tmp, reg(z), reg(y); sltiu reg(x), tmp, 1 - not (z < y)

Eq	<code>sub tmp, reg(y), reg(z); sltiu reg(x), tmp, 1</code> — 即 seqz
Neq	<code>sub tmp, reg(y), reg(z); sltu reg(x), zero, tmp</code> — 即 snez
And/Or	逻辑短路已由 IR translator 处理, 这里不应该出现, 可以 assert

关键技巧: 关系运算 (Lt/Gt/Le/Ge/Eq/Neq) 需要产生 0 或 1 的值。RISC-V 的 `slt` 配合 `sltiu` (即 seqz) 能组合出所有关系。
先试试 `selectUnary`, 再做 `selectBinary`?

25.1s



杨思行

06/09 17:11

selectBinary我没有什么好的思路, 你说的new_temp()我也不知道怎么实现合适, 请给我点提示



deepseek-v4-pro

06/09 17:11

执行过程: 1 条消息

两个问题分开来讲。

new_temp() 怎么实现

它就是一个生成唯一临时变量名的辅助函数。思路:

- 在 `inst_selector.hpp` 的 `InstSelector` 类里加一个成员变量 `int temp_counter = 0;`
- `new_temp()` 每次调用返回 `"tmp" + std::to_string(temp_counter++)`, 得到 `@tmp0`、`@tmp1`、`@tmp2` ...

用 `@` 前缀是因为用户变量名不可能以 `@` 开头, 不会冲突。实现只需要一行返回语句。

参考模板里已有的类似模式——`cfg_builder.cpp` 第 76 行的 `new_label()` 就是用一个 `static int` 计数器生成 `"LN0"`、`"LN1"` ..., 思路完全一样, 你可以直接照搬。

selectBinary 怎么组织

关键难点不是算法, 是操作数可能是立即数。IR 的 `Binary` 节点里 `node->y` 和 `node->z` 是 `std::string`, 可能是:

- `"T5"` — 普通变量名
- `"#3"` — 立即数 (以 `#` 开头)

思路分三步:

第一步: 判断 + 标准化操作数。如果某个操作数以 `#` 开头, 先用 `Li` 把它加载到一个临时虚拟寄存器里, 然后把这个临时寄存器的名字当作新的操作数名。伪代码:

Code

复制

```
y_name = node->y
z_name = node->z

if y_name 以 '#' 开头:
    tmp = new_temp()
    emit Li Reg(tmp), stoi(y_name去掉#)
```

杨思行

selectBinary我没有什么好的思路，你说的new_temp()我也不知道怎么实现合适，请给我点提示

```
z_name = node->z

if y_name 以 '#' 开头:
    tmp = new_temp()
    emit li Reg(tmp), stoi(y_name去掉#)
    y_name = tmp

if z_name 以 '#' 开头:
    tmp = new_temp()
    emit li Reg(tmp), stoi(z_name去掉#)
    z_name = tmp
```

std::stoi(s.substr(1)) 可以把 "#3" 去掉 # 转成 int 3。

第二步：按 node->op 分派。大部分算术运算直接用 Arith::create 即可：

- Add → Arith::Op::Add
- Sub → Arith::Op::Sub
- Mul → Arith::Op::Mul
- Div → Arith::Op::Div
- Mod → Arith::Op::Rem

第三步：关系运算需要产生 0/1 值。以 Lt（小于）为例最简单：

Code

复制

```
x = y < z  →  slt reg(x), reg(y), reg(z)  // x = 1 if y < z, else x = 0
```

Gt 交换操作数就行。Le/Ge 需要两步（用 not(opposite) 的思路），例如 $x = y \leq z$ 等价于 $x = !(z < y)$ ：

Code

复制

```
slt tmp, reg(z), reg(y)  // tmp = 1 if z < y (即 y > z)
sltiu reg(x), tmp, 1     // x = (tmp == 0), 即 seqz
```

Eq 和 Neq 也是两步：先 sub 求差，再 sltiu（即 seqz）或 sltu（即 snez）判断差是否为 0。

总结

你需要新增的东西：

位置	内容
<code>inst_selector.hpp</code>	<code>int temp_counter = 0;</code> 成员变量 + <code>std::string new_temp();</code> 声明
<code>inst_selector.cpp</code>	<code>new_temp()</code> 实现（一行）
<code>inst_selector.cpp</code>	<code>selectBinary</code> 实现（三步结构）
<code>inst_selector.cpp</code>	<code>selectUnary</code> 实现（switch UnaryOp）

先试试？哪里卡住了再问。



19.4s



杨恩行

selectBinary我没有什么好的思路，你说的new_temp()我也不知道怎么实现合适，请给我点提示

06/09 17:17

new_temp()应该怎么写我不大会：二元操作数的两个操作数应该是什么类型的我不是很确定



deepseek-v4-pro

06/09 17:17

执行过程：1 条消息

new_temp() 写法

完全照抄 `c_cfg_builder.cpp` 里已有的 `new_label()`，改个前缀就行：

`inst_selector.hpp` — 在 `private` 区域加：

C++

复制

```
int temp_counter = 0;
std::string new_temp();
```

`c_inst_selector.cpp` — 实现：

C++

复制

```
std::string InstSelector::new_temp() {
    return "@tmp" + std::to_string(temp_counter++);
}
```

这就完了。每次调用返回 `@tmp0`、`@tmp1`、`@tmp2` ...

Binary 操作数类型

在 `ic.hpp` 里（第 54-75 行），`Binary` 的定义是：

C++

复制

```
class Binary : public Node {
public:
    std::string x;    // 结果变量，如 "T5"
    std::string y;    // 左操作数，可能是 "T3" 或 "#1"
    BinaryOp op;      // 运算符，如 BinaryOp::Add
    std::string z;    // 右操作数，可能是 "T4" 或 "#3"
};
```

`y` 和 `z` 都是 `std::string`。你需要判断它是不是立即数——规则很简单：看第一个字符是不是 `'#'`。

C++

复制

```
bool y_is_imm = (node->y[0] == '#');
bool z_is_imm = (node->z[0] == '#');
```

- 如果是 `"T3"` → 不是立即数 → 直接用 `ASM::Reg(node->y)` 作为操作数
- 如果是 `"#3"` → 是立即数 → 先 `li` 加载到临时寄存器，然后把临时寄存器名当作操作数



具体来说，处理完后你手里有两个干净的变量名 `y_name` 和 `z_name`（都确认不是立即数了），然后：

```
code.push_back(ASM::Arith::create(
    ASM::Reg(node->x),      // rd: 结果
    ASM::Reg(y_name),      // rs1: 左操作数
    ASM::Reg(z_name),      // rs2: 右操作数
    op                     // Arith::Op::Add 等
));
```

ASM::Reg 的构造函数只接受 std::string，所以你传变量名字符串进去就好。

思路清楚了没？先写 selectUnary 热热身，再写 selectBinary 会更顺。

14.2s



杨思行

06/09 17:35

selectUnary已经写好了，另一个确实没写出来，请你指导我一下



deepseek-v4-pro

06/09 17:36

执行过程：2次工具调用，3条消息

看到你的代码了。selectBinary 写到一半卡住了，selectUnary 也有几个问题。我一个一个说。

先修 selectUnary

三个问题：

1. UnaryOp::NEG → 小写了，应该用 Neg。看 common.hpp 第 58 行枚举定义。
2. ASM::Sub::create 不存在。ASM 里只有一个 Arith 类，通过 Arith::Op 区分加减乘除。取负应该是：

C++

复制

```
ASM::Arith::create(Reg(node->x), Reg::zero, Reg(node->y), ASM::Arith::Op::Sub)
```

注意 Reg::zero，不是 Reg("zero") —— 它已经是预定义的静态常量。

3. ASM::Xor::create 也不存在。逻辑非 !y 应该用 sltiu (set-less-than-immediate-unsigned)：

Code

复制

```
sltiu reg(x), reg(y), 1  → x = (y < 1) ? 1 : 0  → x = (y==0)
```

这就是 seqz 伪指令的效果。对应的代码：

C++

复制

```
ASM::ArithImm::create(Reg(node->x), Reg(node->y), 1, ASM::ArithImm::Op::Sltu)
```

再写 selectBinary

你卡住的核心问题：当检测到操作数是立即数时，怎么把它转成变量名？ ↓ 很简单：

Rust

复制

selectUnary已经写好了，另一个确实没写出来，请你指导我一下

```
static Reg(x), Reg(y), 1, ASM::ArithImm::Op::Sltu);
```

这就是 seqz 伪指令的效果。对应的代码：

C++

复制

```
ASM::ArithImm::create(Reg(node->x), Reg(node->y), 1, ASM::ArithImm::Op::Sltu)
```

再写 selectBinary

你卡住的核心问题：当检测到操作数是立即数时，怎么把它转成变量名？思路很简单：

Rust

复制

```
y_name = node->y // 从 IR 拿来原始字符串
z_name = node->z

if y_name[0] == '#':
    tmp = new_temp() // "@tmp0"
    val = stoi(y_name.substr(1)) // "#3" → 3
    code.push_back(Li::create(Reg(tmp), val)) // li @tmp0, 3
    y_name = tmp // 把 "@tmp0" 当作新的操作数名

if z_name[0] == '#':
    // 同理
```

处理完后，y_name 和 z_name 都是干净的变量名了。然后做 op 分派：

CSS

复制

```
switch (node->op):
    Add → Arith::create(Reg(x), Reg(y_name), Reg(z_name), Arith::Op::Add)
    Sub → Arith::create(Reg(x), Reg(y_name), Reg(z_name), Arith::Op::Sub)
    Mul → Arith::create(...)
    ...
    Lt → Arith::create(Reg(x), Reg(y_name), Reg(z_name), Arith::Op::Slt)
    Gt → Arith::create(Reg(x), Reg(z_name), Reg(y_name), Arith::Op::Slt) // 交换
    Le → 两步：先 slt tmp, z_name, y_name (即 z < y = y > z); 再 sltiu x, tmp, 1 (取反)
    Ge → 两步：先 slt tmp, y_name, z_name; 再 sltiu x, tmp, 1
    Eq → 两步：先 sub tmp, y_name, z_name; 再 sltiu x, tmp, 1 (seqz)
    Neq → 两步：先 sub tmp, y_name, z_name; 再 sltu x, zero, tmp (snez)
```

两步运算的关键：中间结果也需要一个临时虚拟寄存器。用 new_temp() 拿一个就行，然后用完即弃：

C++

复制

```
case BinaryOp::Eq: {
    auto tmp = new_temp();
    code.push_back(Arith::create(Reg(tmp), Reg(y_name), Reg(z_name), Arith::Op::Sub));
    code.push_back(ArithImm::create(Reg(node->x), Reg(tmp), 1, ArithImm::Op::Sltu));
    break;
}
```

先修 selectUnary，然后按这个思路写 selectBinary，有问题随时时间

29.3s

🔍 📄 🔄

↓



杨恩行

06/09 23:13

目前实现的selectBranch和selectIf的逻辑好像是相同的是不是哪里有问题？



deepseek-v4-pro

06/09 23:13

执行过程：5 次工具调用，3 条消息

确实有问题。看出来两个 bug：

Bug 1: dispatch 里写了 `SELECT_NODE(Branch)`

第 50 行：

C++

复制

```
SELECT_NODE(Branch)
```

`Branch` 是 `ASM` 指令类（在 `asm.hpp` 里），不是 `IR` 节点。`IR` 里条件分支的节点叫 `If`（定义在 `ir.hpp` 第 127 行）。`SELECT_NODE(Branch)` 会展开成 `dynamic_pointer_cast<IR::Branch>`，但 `IR::Branch` 不存在。

应该改成：

C++

复制

```
SELECT_NODE(If)
```

Bug 2: `selectIf` 里用了未定义的 `x_name` / `y_name`

第 327 行开始：

C++

复制

```
if(x_name[0] == '#') { // x_name 从哪来的？
```

你在 `selectBinary` 里写的是：

C++

复制

```
auto y_name = node->y; // - 先声明
auto z_name = node->z;
```

但 `selectIf` 漏了这两行。在 `selectIf` 开头加上：

C++

复制

```
auto x_name = node->x;
auto y_name = node->y;
```

Bug 3: switch 漏了 `Gt` 和 `Le`

`selectIf` 的 switch 只有 `Eq/Neq/Lt/Ge` 四种，但 `BinaryOp` 跟 `If` 的 ↓ 表达式一样有 6 种比较。缺了：

Code

复制

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

杨恩行

目前实现的selectBranch和selectIf的逻辑好像是相同的是不是哪里有问题?

Bug 2: selectIf 里用了未定义的 x_name / y_name

第 327 行开始:

```
C++  
  
if(x_name[0] == '#') { // x_name 从哪来的?
```

你在 selectBinary 里写的是:

```
C++  
  
auto y_name = node->y; // ← 先声明  
auto z_name = node->z;
```

但 selectIf 漏了这两行。在 selectIf 开头加上:

```
C++  
  
auto x_name = node->x;  
auto y_name = node->y;
```

Bug 3: switch 漏了 Gt 和 Le

selectIf 的 switch 只有 Eq/Neq/Lt/Ge 四种, 但 BinaryOp 跟 If 的条件表达式一样有 6 种比较。缺了:

```
Code  
  
Gt → Blt (交换操作数)  
Le → Bge (交换操作数)
```

加上这两个 case, 最后一条 Branch::create 要配合交换——Gt 和 Le 需要把 rs1 / rs2 颠倒:

```
C++  
  
case BinaryOp::Gt:  
    code.push_back(ASM::Branch::create(ASM::Reg(y_name), ASM::Reg(x_name),  
                                        node->label, ASM::Branch::Op::Blt));  
    return code;  
case BinaryOp::Le:  
    code.push_back(ASM::Branch::create(ASM::Reg(y_name), ASM::Reg(x_name),  
                                        node->label, ASM::Branch::Op::Bge));  
    return code;
```

普通情况 (Eq/Neq/Lt/Ge) 正常传参即可:

```
C++  
  
code.push_back(ASM::Branch::create(ASM::Reg(x_name), ASM::Reg(y_name),  
                                    node->label, op));
```

修这三处就行: dispatch 改成 SELECT_NODE(If)、补上 x_name/y_name 声明、补 Gt/Le 两个 case。

31.7s



杨恩行

06/09 23:17

selectIf和Branch有什么关系



deepseek-v4-pro

06/09 23:17

执行过程: 1 条消息

selectIf 和 Branch 是不同层的东西，模式和前面的完全一致：

C++

复制

IR 层		指令选择函数		ASM 层
IR::If	→	selectIf()	→	ASM::Branch
IR::Binary	→	selectBinary()	→	ASM::Arith
IR::Load	→	selectLoad()	→	ASM::Load

- IR::If：IR 数据节点，存着 `x`, `op`, `y`, `label`，表示 "IF x op y GOTO label"
- selectIf：翻译函数，把 IR::If 的信息读出来，选择对应的 ASM::Branch::Op
- ASM::Branch：ASM 指令类，输出 `beq rs1, rs2, label` 这样的 RISC-V 指令

你之前混淆是因为把 ASM 层的名字 Branch 当成了 IR 层的名字，写了 `SELECT_NODE(Branch)` 和 `selectBranch(const IR::BranchPtr&)`。IR 里存放条件分支的节点就叫 If，不是 Branch。

12.0s



杨恩行

06/09 23:19

请继续后面的部分



deepseek-v4-pro

06/09 23:19

执行过程: 1 条消息

好。接下来是 selectParam 和 selectDec，指令选择就完整了。这两个都需要先在头文件加声明、dispatch 里加 SELECT_NODE。

selectParam

IR 的 Param 节点（[ir.hpp](#) 第 218-231 行）只有一个字段 `node->x`，没有索引。因为参数按顺序出现在函数体开头，你需要用一个计数器来跟踪是第几个参数。

修改三步：

1. `inst_selector.hpp` 加成员变量 `int param_index = 0;`
2. `select(FunctionPtr& func)` 里进入新函数时重置 `param_index = 0;`
3. `selectParam` 逻辑：

C++

复制

3. selectParam 选福。

杨思行

请继续后面的部分

```
int k = param_index++;  
if (k < 8) // 前 8 个参数在 a0-a7  
    mv Reg(x), Reg("a" + to_string(k))  
else // 第 9 个起在栈上, fp + 8 开始  
    lw Reg(x), 8 + (k-8)*4(fp)
```

selectDec

IR 的 Dec 节点 ([ir.hpp](#) 第 252-269 行) 有 `node->x` (目标变量) 和 `node->k` (字节数)。需要用 `current_func->alloc_temp(node->k)` 在栈上分配空间, 然后:

C++

复制

```
int offset = current_func->alloc_temp(node->k);  
// Reg(x) = sp + offset -> addi Reg(x), sp, offset  
ArithImm::create(Reg(x), Reg::sp, offset, ArithImm::Op::Add)
```

但: `alloc_temp` 还没实现——它在 `control_flow.cpp` 里还是个 `#warning` 空壳。所以需要先实现它, 就一行:

C++

复制

```
int Function::alloc_temp(int size) {  
    int offset = temp_stack_size;  
    temp_stack_size += size;  
    return offset;  
}
```

同理 `alloc_reg`:

C++

复制

```
int Function::alloc_reg(int size) {  
    int offset = reg_stack_size;  
    reg_stack_size += size;  
    return offset;  
}
```

建议顺序: 先 `alloc_temp/alloc_reg` → 再 `selectDec` → 最后 `selectParam`。从哪里开始?

20.8s



杨思行

06/09 23:37

请你检查一下我们实现的是否正确



deepseek-v4-pro

06/09 23:38

执行过程: 3 次工具调用, 2 条消息

发现 7 个问题, 有些是编译错误, 有些是逻辑错误。

问题 1 — `selectDec`: `dispatch` 里没注册

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)



deepseek-v4-pro

06/09 23:38

执行过程: 3 次工具调用, 2 条消息

发现 7 个问题, 有些是编译错误, 有些是逻辑错误。

问题 1 — selectDec: dispatch 里没注册

第 44-59 行缺少 `SELECT_NODE(Dec)` 和 `SELECT_NODE(Param)`。不加的话跑起来就会 `assert(false && "Unknown IR node type")`。

问题 2 — selectDec: 类型名大小写不一致

.hpp 第 39 行声明 `const IR::DecPtr&` (大写 P), 但 `ir.hpp` 里的定义是 `Decptr` (小写 p)。`.cpp` 也用小写。必须统一。

问题 3 — selectDec: Arith::create 两个错误

第 338 行 `Arith::create` — 拼写错误 (`Arithm`), 而且不应该用 `Arith::create`。这里操作数是 `sp` + 一个整数 `offset`, 应该用 `ArithImm::create` (带立即数的加), `Op` 也应该是 `ArithImm::Op::Add`。

问题 4 — selectParam: 方向反了

第 349 行:

C++

复制

```
Mv::create(Reg("a" + ...), Reg(node->x)) // mv a{k}, x ← 错
```

`Param` 是接收参数——值在 `a{k}` 里, 要搬到 `x`。方向是:

C++

复制

```
Mv::create(Reg(node->x), Reg("a" + ...)) // mv x, a{k} ← 对
```

问题 5 — selectParam: off-by-one 用错变量

第 348 行判断 `param_index < 8`, 但此时 `param_index` 已经 ++ 了。第 8 个参数 (`k=7`, 应在 `a7`) 会因为 `param_index=8 < 8` 而错误走栈分支。应该判断 `k < 8`。

问题 6 — selectParam: 栈参数应 Load 不是 Store

第 351 行 `Store::create(Reg(x), Reg("sp"), ...)` 是把 `x` 写到栈上——方向反了。栈参数是调用者在 `call` 前推到栈上的, 接收方应该从栈读出:

C++

复制

```
// 第一个栈参数(k=8)在 fp+8 的位置  
Load::create(Reg(node->x), Reg::fp, 8 + (k - 8) * 4)
```

问题 7 — selectParam: param_index 没重置

需要在 `select(FunctionPtr& func)` (第 18 行) 函数体开头加 `param_index = 0;`, 否则切换到下一个函数时计数器不会归零。

总结:

位置

问题



dispatch

SELECT_NODE(Dec) 和 SELECT_NODE(Param)

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

第 348 行判断 `param_index < 0`，但此时 `param_index` 已经 ++ 了。第 0 个参数 ($k=0$ ，应往 `a[0]` 去) 因为 `param_index=0 < 0` 而错误走栈分支。应该请你检查一下我们实现的是否正确

第 351 行 `Store::create(Reg(x), Reg("sp"), ...)` 是把 `x` 写到栈上——方向反了。栈参数是调用者在 `call` 前推到栈上的，接收方应该从栈读出：

```
C++  
  
// 第一个栈参数(k=8)在 fp+8 的位置  
Load::create(Reg(node->x), Reg::fp, 8 + (k - 8) * 4)
```

问题 7 — `selectParam` : `param_index` 没重置

需要在 `select(FunctionPtr& func)` (第 18 行) 函数体开头加 `param_index = 0;`，否则切换到下一个函数时计数器不会归零。

总结：

位置	问题
dispatch	缺 <code>SELECT_NODE(Dec)</code> 和 <code>SELECT_NODE(Param)</code>
.hpp:39	<code>DecPtr</code> → <code>Decptr</code>
selectDec:338	<code>Arithm::create</code> → <code>ArithImm::create</code> ，Op 用 <code>ArithImm::Op::Add</code>
selectParam:348	<code>param_index < 8</code> → <code>k < 8</code>
selectParam:349	Mv 参数反了: <code>(Reg(x), Reg(a(k)))</code>
selectParam:351	<code>Store</code> → <code>Load</code> ，用 <code>Reg::fp</code> 不是 <code>Reg("sp")</code> ，offset 是 <code>8 + (k-8)*4</code>
select():18	缺 <code>param_index = 0;</code>

要帮你直接改吗？

46.1s



杨思行
06/09 23:44

`param_index` 从 0 开始是否本来就应该判断的是 `param_index < 8`



deepseek-v4-pro
06/09 23:44

执行过程：1 条消息

不对，用 `param_index < 8` 恰好有一个 off-by-one 错误。推演一下：

```
Code  
  
param_index 起始 = 0  
  
第 1 个参数: k = 0, param_index → 1, 判断 1 < 8 → a0 ✓  
第 2 个参数: k = 1, param_index → 2, 判断 2 < 8 → a1 ✓  
...  
第 8 个参数: k = 7, param_index → 8, 判断 8 < 8 → 走栈 ✗
```

第 8 个参数 `k=7` 应该进 `a7`，但因为判断的是 ++ 后的 `param_index=8`，`8 < 8` 为 false，被错误地当成栈参数了。正确做法是判断 `k` 本身：`k < 8`。

13.8s

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

C++

复制

```
// 第一个栈参数(k=8)在 fp+8 的位置
Load::create(Reg(node->x), Reg::fp, 8 + (k - 8) * 4)
```

问题 7 — selectParam: param_index 没重置

需要在 select(FunctionPtr& func) (第 18 行) 函数体开头加 param_index = 0; , 否则切换到下一个函数时计数器不会归零。

总结:

位置	问题
dispatch	缺 SELECT_NODE(Dec) 和 SELECT_NODE(Param)
.hpp:39	DecPtr → Decptr
selectDec:338	Arithm::create → ArithImm::create, Op 用 ArithImm::Op::Add
selectParam:348	param_index < 8 → k < 8
selectParam:349	Mv 参数反了: {Reg(x), Reg(a(k))}
selectParam:351	Store → Load, 用 Reg::fp 不是 Reg("sp"), offset 是 8 + (k-8)*4
select():18	缺 param_index = 0;

要帮你直接改吗?

46.1s



杨思行

06/09 23:44

param_index从0开始是否本来就应该判断的是param_index<8



deepseek-v4-pro

06/09 23:44

执行过程: 1 条消息

不对, 用 param_index < 8 恰好有一个 off-by-one 错误。推演一下:

Code

复制

```
param_index 起始 = 0

第 1 个参数: k = 0, param_index → 1, 判断 1 < 8 → a0 ✓
第 2 个参数: k = 1, param_index → 2, 判断 2 < 8 → a1 ✓
...
第 8 个参数: k = 7, param_index → 8, 判断 8 < 8 → 走栈 ✗
```

第 8 个参数 k=7 应该进 a7, 但因为判断的是 ++ 后的 param_index=8, 8 < 8 为 false, 被错误地当成栈参数了。正确做法是判断 k 本身: k < 8。

13.8s



帮我直接改掉这 7 个问题



杨思行

06/09 23:47

```
// 第一个栈参数(k=8)在 fp+8 的位置
Load::create(Reg(node->x), Reg::fp, 8 + (k - 8) * 4)为什么是8 + (k - 8) * 4
```



deepseek-v4-pro

06/09 23:48

› 执行过程: 1 条消息

因为函数调用栈帧的布局。推演一下 `call` + prologue 的完整过程:

调用者 (caller) 在 `call` 之前:

Code

复制

高地址

...

arg9 (第9个参数) → old_sp + 4

arg8 (第8个参数) → old_sp + 0

执行 `call func`: `ra` 被硬件压栈 → `sp = old_sp - 4`

我们 prologue 里:

Code

复制

```
addi sp, sp, -frame_size    → sp = old_sp - 4 - frame_size
sw  ra,  frame_size-4(sp)
sw  fp,  frame_size-8(sp)
addi fp, sp, frame_size-8    → fp = sp + frame_size - 8 = old_sp - 8
```

所以 prologue 之后:

Code

复制

```
fp + 8 = old_sp            → arg8 (第一个栈参数)
fp + 12 = old_sp + 4       → arg9
...
fp + 8 + (k-8)*4           → arg(k)
```

那个 8 就是 `ra` (4 字节) + `fp` (4 字节) 占的空间, 即从 `fp` 往上跳过这两个保存值才能到达调用者压参的起始位置。

12.4s



杨思行

杨思行
那我们进入下一阶段

杨思行
06/10 09:39

我现在实现的有点问题请你帮我改进一下



deepseek-v4-pro
06/10 09:40

执行过程: 2 次工具调用, 2 条消息

思路方向没错, 但实现上有几个问题。逐一来说:

问题 1: `reg in reg_map` 不是 C++ 语法

第 28 行和第 41 行, `in` 是 Python。C++ 应该写 `reg_map.find(reg) != reg_map.end()`。

问题 2: 类型混乱

`alloc_temp(4)` 返回 `int` (sp 偏移量), 但下一行 `temp = available_regs.begin()->name` 又赋成了 `string`。这是两种不同的东西:

- 栈偏移量 (`int`): 变量存在栈上哪个位置
- 寄存器名 (`string`): 临时用的物理寄存器

你需要分别用两个变量存。

问题 3: `lw/sw` 的 `offset` 硬编码为 4

第 36 和 49 行, `offset` 总是 4——但每个虚拟寄存器分配在不同的栈位置, `offset` 应该各不相同。

问题 4: 方法不对——不应边迭代边插入

在遍历 `block->asm_code` 的同时往里面插指令, 会导致迭代器失效。

正确做法: 构建一个新的 `std::list<InstPtr> new_code`, 逐条处理原始指令, 展开后放入 `new_code`, 最后替换。

正确的朴素 spill-all 流程

Rust

复制

```
第1步: func->alloc_reg(8) // ra + fp

第2步: 收集所有虚拟寄存器名
遍历所有指令的 get_uses() + get_defs()
如果 !reg.is_phys(), 收集 reg.name

第3步: 为每个虚拟寄存器分配栈槽
map<string, int> vreg_offset; // 变量名 -> sp偏移
for each name: vreg_offset[name] = func->alloc_temp(4)

第4步: 展开每条指令
ASM::Reg scratch[] = {t0, t1, t2};

for each inst in 原始代码:
    // 4a. 处理 uses: 前插 lw
    uses = inst->get_uses()
    for each virtual use (按顺序):
        sr = scratch[scratch_idx++ % 3]
```



输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

杨思行

我现在实现的有点问题请你帮我改进一下

```
sr = scratch[scratch_idx++ % 3]
new_code += lw sr, vreg_offset[name](sp)
use_map[vreg] = sr

inst->replace_uses(use_map)

// 4b. 处理 defs: 替换 + 后插 sw
defs = inst->get_defs()
for each virtual def:
    sr = scratch[scratch_idx++ % 3]
    new_code += instruction (替换后的)
    new_code += sw sr, vreg_offset[name](sp)

// 如果没有虚拟 def, 直接 new_code += inst

第5步: block->asm_code = new_code
func->reg_map = {} // 空—虚拟寄存器已在代码中消除了
```

关键点: `use_map` 和 `def_map` 是 `ASM::RegMap (map<Reg, Reg>)`, 把虚拟寄存器映射到临时物理寄存器 `t0/t1/t2`。`replace_uses` 和 `replace_defs` 会按这些映射替换指令内的寄存器引用。

对照这个流程重写 `allocate(FunctionPtr& func)`, 有问题随时间。

40.6s



杨思行

06/10 09:42

请你告诉我具体第二步怎么实现



deepseek-v4-pro

06/10 09:42

> 执行过程: 1 条消息

第 2 步很简单, 就是遍历函数里所有指令的 `get_uses()` 和 `get_defs()`, 把非物理寄存器的名字收集到一个 `set` 里:

```
C++
std::set<std::string> all_vregs;

for (auto& block : func->blocks) {
    for (auto& inst : block->asm_code) {
        for (auto& reg : inst->get_uses()) {
            if (!reg.is_phys()) {
                all_vregs.insert(reg.name);
            }
        }
        for (auto& reg : inst->get_defs()) {
            if (!reg.is_phys()) {
                all_vregs.insert(reg.name);
            }
        }
    }
}
```

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

请你告诉我具体第二步怎么实现

三层循环：

- 外层：遍历函数的所有基本块
- 中层：遍历每个基本块里的 ASM 指令
- 内层：遍历每条指令的 use 集合和 def 集合

用 `std::set` 而不是 `std::vector` 是因为同一变量可能在多条指令里出现，set 自动去重。`is_phys()` 返回 true 的话说明是 `t0`、`a0`、`sp` 这类物理寄存器，不需要分配栈空间，跳过即可。

拿到 `all_vregs` 之后，第 3 步就是为每个名字分配栈槽：

C++

复制

```
std::map<std::string, int> vreg_offset;
for (auto& name : all_vregs) {
    vreg_offset[name] = func->alloc_temp(4);
}
```

`name` → `sp`偏移 的映射就建好了。后面第 4 步展开指令时，用 `vreg_offset[name]` 就能拿到这个变量的 `sp` 偏移量。

9.8s



杨思行

06/10 10:18

我写的可能不是很好，你看一下然后给出你认为正确的写法



deepseek-v4-pro

06/10 10:19

执行过程：1 次工具调用，2 条消息

看了你的代码，整体结构方向对了，但有几个关键问题：

你的版本的问题

1. `use_map` 跨指令累积（第 79-87 行）：没有每指令重置，上一指令的映射会影响后面的 `replace_uses`
2. 没有 `def_map` 也没有 `replace_defs`：指令的 `def` 寄存器仍然是虚拟的，没有被替换成物理寄存器
3. 有 `def` 时才 `push inst`（第 95 行）：导致无 `def` 的指令（`call`, `j`, `ret`, `label` 等）全部丢失
4. 多个 `def` 会重复 `push inst`：def 循环里每次都 `push inst`，如果一条指令有 2 个 `def`，`inst` 被添加两次
5. `new_code` 跨 `block` 混一起：应该每个 `block` 独立构建 `new_code`，最后赋值给 `block->asm_code`
6. 第 79 行：`ASM::RegMaMap` 拼写错误，应该是 `ASM::RegMap`

正确写法

C++

复制

```
void RegAllocator::allocate(FunctionPtr& func) {
    func->alloc_reg(8); // ra + fp

    // 收集所有虚拟寄存器
    std::set<std::string> virtual_regs;
    for (auto& block : func->blocks) {
        for (auto& inst : block->asm_code) {
```



入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)


```
ASM::Reg scratch[] = {ASM::Reg::t0, ASM::Reg::t1, ASM::Reg::t2};

for (auto& block : func->blocks) {
    ASM::Code new_code;    // 每个 block 独立构建
    for (auto& inst : block->asm_code) {
        int si = 0;
        ASM::RegMap use_map, def_map;

        // 处理 uses: 指令前插入 lw
        for (auto& reg : inst->get_uses()) {
            if (!reg.is_phys()) {
                ASM::Reg sr = scratch[si++ % 3];
                new_code.push_back(
                    ASM::Load::create(sr, ASM::Reg::sp, vreg_offset[reg.name]));
                use_map[reg] = sr;
            }
        }
        inst->replace_uses(use_map);

        // 处理 defs: 映射到 scratch 寄存器
        for (auto& reg : inst->get_defs()) {
            if (!reg.is_phys()) {
                ASM::Reg sr = scratch[si++ % 3];
                def_map[reg] = sr;
            }
        }
        inst->replace_defs(def_map);

        // 指令本身 (只 push 一次)
        new_code.push_back(inst);

        // 指令后插入 sw (把结果存回栈)
        for (auto& [vreg, sr] : def_map) {
            new_code.push_back(
                ASM::Store::create(ASM::Reg::sp, sr, vreg_offset[vreg.name]));
        }
        block->asm_code = new_code;    // 替换当前 block 的代码
    }

    func->reg_map = {};    // 虚拟寄存器已全部消除
}
```

关键区别：

	你的版本	正确版本
use/def map	跨指令共享一个	每条指令重置，独立构建
replace_defs	没调用	每条指令都调用
inst push	只有在 def 时 push	无条件 push 一次
new_code	全局一个	每个 block 一个
指令丢失	call/jump/label 全丢	全部保留



杨思行

06/10 10:40

Markdown

复制

****参数溢出空间****: 如果传递给函数的参数超过寄存器能容纳的数量(例如, 超过`a0-a7`的8个参数), 多余的参数会存储在栈帧中能否展示一下这一部分我们在代码中



deepseek-v4-pro

06/10 10:41

执行过程: 1 条消息

当前代码在两个地方体现了参数超出 8 个的处理——但调用方有遗漏。

被调用方 (selectParam) — 已正确处理

C++

复制

```
// inst_selector.cpp
int k = param_index++;
if (k < 8) {
    // 前 8 个: 从寄存器接收
    mv reg(x), a{k}
} else {
    // 第 9 个起: 从调用者的栈帧读取
    lw reg(x), 8 + (k-8)*4(fp)
}
```

对应在栈帧中的位置 (从被调用者视角看):

Code

复制

```
      高地址
      ...
      arg9      ~ fp + 12
      arg8      ~ fp + 8   ~ 第一个栈参数
      ----- fp -----
      saved ra   ~ fp + 4
      saved fp   ~ fp + 0
      ...
      sp
      低地址
```

`fp + 8` 跳过了 `saved fp` (4B) + `saved ra` (4B), 正好指到调用者压参的位置。用 `8 + (k-8)*4` 算出 `arg(k)` 的偏移。

调用方 (selectArg) — 缺 $k \geq 8$ 的处理

当前只写了:

C++

复制

```
// k < 8 的: mv a{k}, reg(x) ~ OK
// k ≥ 8 的: ~ 漏了!
```



对应在线框中的位置（从被调用者视角看）：

Code 复制

```
高地址
...
arg9      ← fp + 12
arg8      ← fp + 8   ← 第一个栈参数
-----fp-----
saved ra   ← fp + 4
saved fp   ← fp + 0
...
sp
低地址
```

`fp + 8` 跳过了 `saved fp` (4B) + `saved ra` (4B)，正好指到调用者压参的位置。用 `8 + (k-8)*4` 算出 `arg(k)` 的偏移。

调用方（`selectArg`）— 缺 `k ≥ 8` 的处理

当前只写了：

C++ 复制

```
// k < 8 的: mv a{k}, reg(x) ← OK
// k ≥ 8 的:      ← 漏了!
```

`k ≥ 8` 时，应该在 `call` 之前把参数写到自己的 `outgoing` 参数区（`sp` 下方），对应栈帧图里底部的 `Outgoing Arguments` 区域：

Code 复制

```
sp -----
arg8 (out) ← (0)*4(sp)
arg9 (out) ← (1)*4(sp)
...
低地址
```

需要补充的代码：

C++ 复制

```
} else {
    // k ≥ 8: 写到 outgoing argument area
    code.push_back(ASM::Store::create(
        ASM::Reg::sp,           // 基址
        ASM::Reg(node->x),      // 要传的值
        (node->k - 8) * 4));     // offset
}
```

不过这里有个隐含问题：`sp` 是否已经预留了足够的 `outgoing` 参数空间？朴素方案里可以让 `emitter` 在 `prologue` 中根据整个模块里最大的 `outgoing` 参数数量统一预留，或者干脆不做——因为测试用例里参数超过 8 个的情况（`spill_test.sy` 有 35 个参数！），需要在 `alloc_temp` / 帧大小计算时考虑这一点。对于先跑通基本测试来说，可以先把调用方的 `selectArg` 补上 `k ≥ 8` 的 `store` 逻辑。

总结：`selectParam`（收参方）是正确的，`selectArg`（传参方）缺 `k ↓` 的处理。要现在补上吗？

1m 5s 复制 点赞 分享

输入消息... (Enter 发送, Shift+Enter 换行, @ 引用文件, / 调用 Skill, # 调用 MCP, & 引用会话)

Step1已经改好了如果不对你直接修复就可以了，接着请你知道我完成step2



杨思行

06/10 11:21

3. 遍历 block 输出指令：

把 `for (i = 0; i < blocks.size() - 1; i++)` 改成 `for (i = 0; i < blocks.size(); i++)`。不再跳过 exit block，因为 `func.ret:` 标签在里面，`GOTO func.ret / j func.ret` 需要它为什么你是这样认为的



deepseek-v4-pro

06/10 11:21

执行过程：1 条消息

你说得对，我之前的建议有问题。重新梳理一下：

原模板注释说：

“为了方便 emit epilogue，这里忽略 exit block”

这个设计是对的。因为 exit block 里只有两样东西：

1. `LABEL func.ret:` — 被指令选择转成 `func.ret:`
2. `RETURN a0` — 被指令选择转成空（`selectReturn` 返回空代码）

所以正确的做法是：保持跳过 exit block，但在 epilogue 里手动输出 `func.ret:` 标签。

Code

复制

```
func_name.ret:           - 由 emitter 输出
    lw  ra, frame_size-4(sp)
    lw  fp, frame_size-8(sp)
    addi sp, sp, frame_size
    ret
```

epilogue 本身就是 exit block 在汇编层的体现——标签 + 栈帧回收 + ret。跳过 exit block（避免重复输出标签）是对的，我之理解错了。

28.8s



杨思行

06/10 11:23

请你看一下这一部分我们实现的是否正确



deepseek-v4-pro

06/10 11:24

执行过程：1 次工具调用，2 条消息

有几个问题：

问题 1 — Raw string 不做变量替换

`R"(...)"` 是原始字符串字面量，里面写 `func_name` 就是字面文本 `func_name`，不会替换成 C++ 变量 `func->name`。所以所有函数会输出一样的标签名 `func_name:`，全部冲突。



问题 2 — `addi fp, sp, frame_size`

3. 遍历 block 输出指令：把 `for (i = 0; i < blocks.size() - 1; i++)` 改成 `for (i = 0; i < blocks.size(); i++)`。不再跳过 exit block，因为 `func.ret:` 标签在里面，`GOTO func.ret / j func.ret` 需要它为什么你是这样认为的

有几个问题：

问题 1 — Raw string 不做变量替换

`R"(...)"` 是原始字符串字面量，里面写 `func_name` 就是字面文本 `func_name`，不会替换成 C++ 变量 `func->name`。所以所有函数会输出一样的标签名 `func_name:`，全部冲突。

问题 2 — `addi fp, sp, frame_size`

应该是 `frame_size - 8`。sp 已经减了 `frame_size`，fp 要指向 saved fp 的位置：

Code

复制

```
fp = sp + frame_size - 8    // = old_sp - 8, 指向 saved fp
```

你写的 `sp + frame_size` 让 `fp = old_sp`，直接回到栈顶了。

问题 3 — 缺 `.text` 指令

`use_venus = false` 时，模块开头没有 `.text`。应该在 Venus 分支之前加一行 `output << ".text" << std::endl;`。

问题 4 — `func_name.ret:` 标签

同样被 raw string 吃掉了，所有函数会共用一个 `func_name.ret:` 标签，出现重复定义。

正确写法（用 `<<` 拼接变量）

C++

复制

```
void ASMEmitter::emit(const FunctionPtr& func) {
    int frame_size = func->temp_stack_size + func->reg_stack_size;
    frame_size = (frame_size + 15) & ~15;
    reg_map = func->reg_map;

    // Prologue
    output << "    .globl " << func->name << std::endl;
    output << func->name << ":" << std::endl;
    output << "    addi sp, sp, -" << frame_size << std::endl;
    output << "    sw ra, " << (frame_size - 4) << "(sp)" << std::endl;
    output << "    sw fp, " << (frame_size - 8) << "(sp)" << std::endl;
    output << "    addi fp, sp, " << (frame_size - 8) << std::endl;

    // Body: 跳过 exit block
    for (size_t i = 0; i < func->blocks.size() - 1; i++)
        emit(func->blocks[i]);

    // Epilogue
    output << func->name << ".ret:" << std::endl;
    output << "    lw ra, " << (frame_size - 4) << "(sp)" << std::endl;
    output << "    lw fp, " << (frame_size - 8) << "(sp)" << std::endl;
    output << "    addi sp, sp, " << frame_size << std::endl;
    output << "    ret" << std::endl;
}
```

核心差异：不用 `R"(...)"`，改用 `<<` 逐个拼接，C++ 变量值才会被正确^{***}入。4 个空格缩进对齐 `emit(InstPtr)` 里的缩进风格。

28.6s